



ITESO, Universidad
Jesuita de Guadalajara

PROJECT 3

ADVANCED TRADING STRATEGIES: DEEP LEARNING

MICROSTRUCTURE AND TRADING SYSTEMS

LUIS FELIPE GÓMEZ ESTRADA

ARANTZA GOMEZ HARO GAMBOA AND JAVIER
ALEJANDRO FAJARDO LÓPEZ

OCTOBER 28, 2025

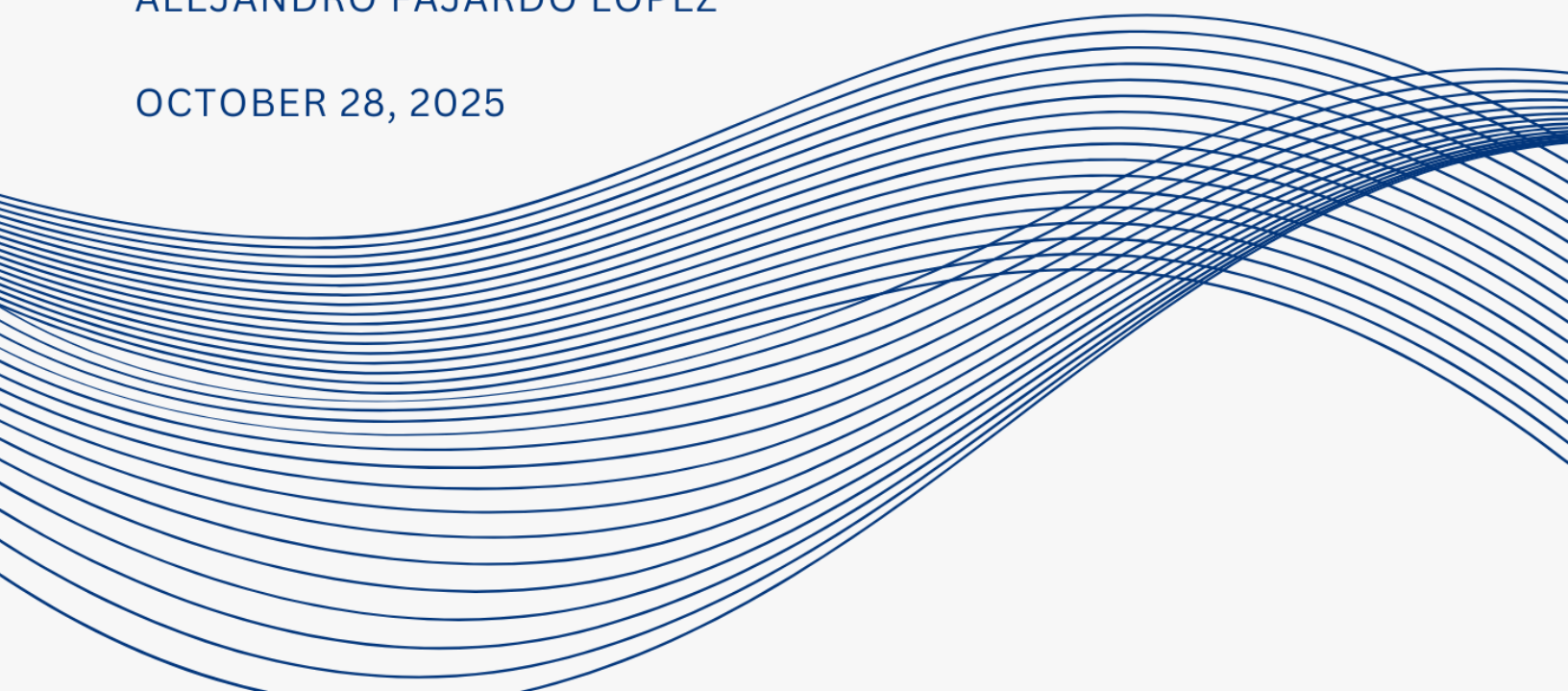


TABLE OF CONTENTS

Introduction.....	4
Strategy Overview	5
Deep Learning	5
Multi-Layer Perceptron and Convolutional Neural Network	6
Expected Advantages and Limitations	7
Feature Engineering.....	8
Indicators	8
Momentum Indicators.....	8
Volatility Indicators	11
Volume Indicators.....	13
Combination of Indicators	15
Normalization.....	16
Fixed Scale Normalization	16
Normalization Relative to Price and Position	16
Z-Score Normalization	17
Min-Max Normalization	18
Normalization of Closing Price.....	18
Class Imbalance Strategy: Threshold Adjustment	18
Target Variable Definition.....	19
Trading Signals.....	19
Threshold Logic for Class Assignment.....	20
Distribution of Signals in Training Data	20
Multi-Layer Perceptron	22
Architecture.....	22
Layer Description	23
Parameters	23
Convolutional Neural Network	25
Convolutional Structure	25
Kernel Size	26
Pooling Strategy	26

Parameters	27
Training Specifications	28
Class Weighting Formula	28
Experiment Tracking	29
Experiment Setup and Logging Structure	29
Parameters Tracked for each Model	29
Metrics Recorded	30
Accuracy	30
Validation Accuracy	30
Validation Loss	30
F1 Score	31
Test Performance	31
Model Comparison: MLP vs. CNN	32
Selected Model	32
Data Drift Monitoring	34
Drift Monitoring Methodology	34
Loading Data and Define Windows	34
K-Test and P-Values	34
Storage and Data Visualization	35
Drift Dashboard	35
Features with Drift	37
Timeline of Drift Periods	38
Backtesting Methodology	39
Position Sizing and Management Rules	39
Data Drift	40
Assumptions and Limitations	40
Best Results and Performance Analysis	41
Equity Curve Plots	41
Train Period	41
Test and Validation Periods	43
Numerical Results	44

Performance Metrics.....	45
Sharpe Ratio	45
Sortino Ratio	45
Maximum Drawdown	46
Calmar Ratio	46
Metric Values.....	47
Trade Statistics	48
Model accuracy vs strategy profitability analysis	49
Comparison	50
Conclusions.....	50
Key Findings and Strategy Viability	50
Model Selection and Performance Summary.....	51
Profitability After Transaction Costs	51
Recommended Improvements or Extensions	52
Bibliography.....	53

Introduction

This project aims to develop a systematic trading strategy based on deep learning models that are designed to predict market signals, using historical market data. By combining feature engineering and time series architectures, such as MLP and CNN, this strategy aims to analyze patterns in the movement of a financial asset and identify potential opportunities.

The approach of this project focuses on the implementation of machine learning engineering to ensure reliability, efficiency and adaptability of the trading strategy. This includes the use of feature engineering, MLFlow, and data drift analysis.

Overall, this project demonstrates how deep learning can be applied to be able to analyze financial markets, providing a solid foundation for developing trading strategies that adapt to evolving market conditions. The complete framework shows how the use of advanced machine learning techniques helps enhance predictive accuracy and improve decision making.

Strategy Overview

Deep Learning

As mentioned previously, this project applies deep learning models to predict market signals to guide trading decisions. Deep learning is suitable for financial time series since it can model complex, non-linear tendencies in market data, being able to identify patterns in price movements and volatility. This type of model uses multi-layer neural networks to capture complex relationships in data. By adding multiple layers of interconnected neurons, deep learning models can identify meaningful and more complex patterns as the data passes through each layer, highlighting what is most relevant for the prediction.

The objective of this strategy is to use deep learning models to generate trading signals that indicate whether to take a long, short or hold position at a given time. Unlike other strategies, deep learning models can learn from historical data by identifying patterns or relationships between technical indicators, market momentum, volatility and prices. Since financial markets are influenced by multiple factors and tend to be very noisy, more traditional approaches fail to capture certain movements or patterns, which is why deep learning architectures are very useful in this context.

Multi-Layer Perceptron and Convolutional Neural Network

This project uses two different deep learning architectures, which are Multi-Layer Perceptron (MLP) and Convolutional Neural Network (CNN). These two models are used together to complement each other in interpreting financial market data.

The MLP model is an artificial neural network that is used for supervised learning in machine learning. The Multilayer Perceptron consists of multiple layers of neurons, where the outputs of the neuron in one layer become the inputs for the next. The different layers which are used in this model are the input layers, hidden layers, and output layers. It is capable of doing classification and regression tasks, which allows it to be used for problems with nonlinear relationships. This model works by adjusting the weights of the network connections to minimize the error in the prediction. Also, it is effective at capturing general patterns across the input features, such as trends in momentum, volatility, and volume. With this said, the MLP works as a great base for the CNN model.

On the other hand, the Convolutional Neural Model is designed to capture more temporal patterns in data. Unlike the MLP, this model is able to identify certain quick relationships that might influence short-term trading decisions, such as temporary trends or sudden changes in volatility. This is done by applying small filters to the input sequence to detect patterns that appear locally and are not general. The CNN layers are sparsely connected, which means that each neuron only processes a small portion of input data, allowing the neural network to focus only on local patterns. These patterns are essential for predicting short-term market movements that may not be apparent from the perspective of the MLP model. Also, the weights of the filters are the same, which allows the CNN to detect the same pattern across the input data. This makes the model more efficient, requiring fewer parameters, and helping prevent overfitting.

Furthermore, by stacking multiple convolutional layers, CNN can learn hierarchies of patterns, allowing it to capture short-term changes in the early layers and more

complex patterns in deeper layers. The use of activation functions, such as ReLU and sigmoid, helps the system learn non-linear relationships, making the network more robust. The pooling layer then pulls the most significant features, summarizing important short-term patterns over time.

The use of multiple architecture helps this strategy have different perspectives, allowing it to analyze financial market information from both a broad and more specific perspective.

Expected Advantages and Limitations

By combining the Multi-Layer Perceptron and the Convolutional Neural Network, this trading strategy will be able to utilize the strengths of both models. The MLP captures stable and broad patterns, while the CNN focuses on more short-term trends and patterns. This complementary approach allows this strategy to adapt to different market conditions, creating a more robust system and allowing a more complete understanding of the market behavior. The robustness of this strategy reduces the chances of missing important trading signals and improves reliability in unpredictable market conditions.

On the other hand, even though this trading strategy uses the strengths of both MLP and CNN, it also has certain limitations. Although the implementation of CNN allows the reduction of overfitting, combining multiple models still requires monitoring to ensure the model performs reliably across different market conditions. The strategy's effectiveness also depends on the quality and consistency of historical data; this is sudden market changes or unusual events may reduce the accuracy of the prediction. Despite these limitations, with careful monitoring and validation, the strategy can maintain a reliable performance while using the strengths of both architectures.

Feature Engineering

Indicators

To effectively predict trading signals, it is essential to provide the model with features that capture key aspects of market behavior. This strategy uses a variety of technical indicators to represent different characteristics of the market, such as momentum, volatility and volume indicators.

Momentum Indicators

Momentum indicators help identify the strength and speed of price movements. In other words, it shows the rate at which a stock's price changes, whether it be a positive or negative change. This type of indicator demonstrates the strength of a stock's price trend, helping decide whether to buy, sell or hold a stock. In this strategy, the momentum indicators that are used are Relative Strength Index, Rate of Change, Exponential Moving Average, and Stochastic Oscillator.

Relative Strength Index

The Relative Strength Index, also known as RSI, is expressed as a value between 0 and 100. It helps identify whether an asset is oversold, when the RSI is below 30, or overbought, when it's over 70, indicating possible changes in the movement of trends. These changes then signal whether it is beneficial to buy or sell an asset. This indicator has a range between 0 and 100. By using different windows, the strategy is able to analyze this indicator and the change in the price in different time periods. The windows used in this strategy are:

- RSI 7: This is a short-term window that reacts quickly to recent changes in the price of an asset. It captures short-term momentum and helps detect rapid shifts in the market, since it is more sensitive to these changes.
- RSI 14: This is the standard window which identifies medium-term trends and patterns, smoothing out some smaller fluctuations. This window provides a

balance between sensitivity and stability in terms of the fluctuations in the price.

- RSI 21: This is a long-term window that captures major and broad trends in the movement of the price. It helps filter out short-term noise and focuses on sustained price movements.

By combining these three RSI windows, this trading strategy can monitor the momentum in multiple timeframes, allowing us to identify both quick market changes and broad patterns.

Rate of Change

The Rate of Change indicator, also known as ROC, is also a momentum indicator that represents an upward momentum with a positive number, and a downtrend with a negative number. It helps identify the speed and strength at which the trend is gaining or losing momentum. This indicator doesn't have a set range, but the values given usually are close to 0, either positively or negatively. The windows used in this strategy are:

- ROC 10: This is a short-term window which is sensitive to recent price changes, allowing the detection of quick changes in momentum. This window signals short-term trading opportunities but may produce false signals.

ROC 20: This is a medium-term window that's provides a balanced sensitivity to price movements. It smooths some of the short-term noise while still reacting to important changes in the trend.

- ROC 45: This is a long-term window that captures a broader perspective of market trends and does not focus on short-term fluctuations. It provides a more stable image of the momentum, allowing us to analyze if price movement is sustained or temporary.

Exponential Moving Average

The Exponential Moving Average, also known as EMA, is an indicator that emphasizes recent data more, making it more responsive to new market information. This allows the indicator to react quickly to changes in current trends and short-term

fluctuations. Although EMA is very responsive to recent data, this may cause false signals due to market noise, which is why it is useful to use different windows. This indicator smooths out the data to be able to analyze clearly the trend that the prices follow. With this said, the value of this indicator is usually close to the price value, since it is a moving average. The windows used for this indicator are:

- EMA 10: This is a short-term window that is highly sensitive to recent data, making it very useful for identifying quick changes in the price.
- EMA 50: This is a long-term window that smooths out short-term fluctuations and provides a clearer view of dominant trends and movement. This window is much less sensitive than EMA 10, which helps reduce noise and focuses on the overall direction of the market.

Stochastic Oscillator

The Stochastic Oscillator is a momentum indicator that helps identify potential trading opportunities by comparing the closing price of an asset with the range between its highs and lows. It measures how strong the market momentum is and how close the price is to its extreme values, which provides insights into whether the asset may be overbought or oversold. This is analyzed with two lines (%K and %D) which represent different levels of sensitivity. The first line is the fast-moving line which reacts quickly to movements, while the second line is slower and helps filter out noise. Like other indicators, the value of Stochastic Oscillator is always between 0 and 100. With this said, the windows used for this indicator are:

- STOCH 12: This is a short-term window that is highly sensitive to recent price changes and is needed to analyze short-term trading opportunities that may be presented.
- STOCH 26: This is a medium-term window that reacts less sharply to recent prices, and it helps to smooth out short-term volatility. This window allows us to confirm if short-term movements represent genuine changes in trend or just temporary movement.
- STOCH 40: This is a long-term window that focuses on a broader perspective of the trends and movements of the prices. It is particularly useful for

confirming dominant market direction and identifying important trading signals.

Importance of Momentum Indicators

Momentum indicators provide valuable insights into the speed and strength of price movements. By analyzing these four different indicators across multiple time windows, the trading strategy captures both short-term fluctuations and long-term trends, allowing a better understanding of market dynamics. This helps improve trading decisions by distinguishing between temporary movements and genuine and consistent trends.

Volatility Indicators

Volatility indicators help measure degree or intensity of price variations over time, reflecting on how stable or unstable an asset's price movement is. A higher volatility implies a greater uncertainty and risk, while a lower volatility suggests a more stable price behavior. By incorporating volatility indicators into the trading strategy, it is possible to better understand market conditions and adjust decision making based on the risks that can be taken. In this strategy, the volatility indicators that are used are Bollinger Bands, Average True Range, and Donchian Channels.

Bollinger Bands

Bollinger Bands are a volatility indicator that is used to identify possible overbought and oversold conditions in the market. This indicator consists of three lines, which are the central line and two outer bands positioned a certain number of standard deviations above and below. These bands expand and contract depending on the volatility of the market. When the bands widen, it indicates higher volatility, which means that the movements in the price tend to be stronger. On the other hand, when the bands contract, it signals lower volatility and a more stable price. Due to this, Bollinger Bands doesn't really have a set range as results, but the value does depend on the price. These bands help analyze when possible new trends might form. The windows that were used for this strategy are:

- BB 15: This is a shorter-term window that is useful for detecting short-term volatility shifts and potential trading opportunities.
- BB 20: This is considered a standard window that provides a greater view of market volatility. It is able to smooth out minor fluctuations and helps identify more stable trends. This window gives insight into a medium-term market behavior.

Average True Range

The Average True Range indicator measures how much an asset price fluctuates. Unlike Bollinger Bands, which uses standard deviations, ATR focuses only on the absolute price changes, which means the result will always be positive and relatively small. In this case, the true range is defined as the greatest value of the current high minus the current low, the absolute value of the current high minus the previous low, and the absolute value of the current low minus the previous close. The ATR is then calculated as a moving average of these true ranges. A higher ATR indicates higher volatility, while a lower ATR suggests a more stable market with lower volatility. In this strategy, the only window that was used is:

- ATR 14: This is a standard window while using ATR, and it captures general volatility patterns, providing a balance between sensitivity and smoothness to avoid overreacting to minor changes in the price.

In this case, if more trading signals were needed, it would be useful to use a shorter window to generate more, but if fewer but more reliable signals were desired, a longer window would be used to smooth out short-term movements.

Donchian Channel

The last volatility indicator that was used is the Donchian Channel indicator. This indicator identifies the highest and lowest values over a specified period, forming a channel around the price. This channel helps highlight changes, trend direction, and potential trading opportunities. When the price moves above the upper line, it indicates a strong upward trend. On the other hand, when the price moves below the lower line, it indicates a downward trend. There is also a middle line that

represents the average of the two extremes. This indicator does not have a set range as a result as it depends on the price of the asset. In this strategy, the window that was used is:

- DON 20: This window, which is the most common number of periods, captures medium-term volatility and trends. This window is able to identify significant price movements, while filtering out some minor movements in the price that don't affect the bigger picture.

Importance of Volatility Indicators

Volatility indicators are essential for understanding how stable or unstable an asset's price is. By using these three indicators, this trading strategy is able to detect periods of high and low volatility, anticipate potential trends, and adjust trading decisions accordingly. This helps manage risk, avoid false signals, and complete momentum indicators to provide a more complete understanding of market behavior.

Volume Indicators

Volume indicators measure the strength of a price movement by analyzing the number of shares that were traded during a specific period. This provides insights into the level of participation in an asset, helping to confirm trends. By incorporating volume indicators into the strategy, it can analyze whether price movements are supported by strong market activity or occur at a lower volume. The volume indicators that were used in this strategy are On Balance Volume, Volume Price Trend, Accumulation Distribution Index, Money Flow Index, and Chaikin Money Flow.

On Balance Volume

The On Balance Volume indicator uses trading volume to anticipate changes in stock prices. This signals potential price movements based on shifts in the volume of the asset stock. The OBV is a cumulative total of volume, both positive and negative, which means the result could be a very big positive or negative number. If the asset closes higher than the previous period, the period's volume is added to the OBV. On the other hand, if the asset closes lower, the volume is subtracted. This indicator

helps determine whether trends are supported by volume or not. In this strategy, OBV is used to confirm momentum and validate trading signals. In this case, there is no window because it is a cumulative measure that considers all historical data, not relying only on fixed periods of time.

Volume Price Trend

The Volume Price Trend indicator, also known as VPT, uses price movements and trading volumes to analyze the strength of a trend. This indicator is a cumulative volume line that either adds or subtracts a multiple of the percentage change in the price's trend and volume. When the price rises, a fraction of the volume proportional to the price change is added to the VPT, and when it is the other way around, it is subtracted. Like the On Balance Volume indicator, VPT does not require a specific window, as it is a cumulative indicator. This also means that the output or result will be either a positive or negative number.

Accumulative Distribution Index

The Accumulative Distribution Index, better known as ADI, is an indicator that uses the correlation price and volume to analyze buying and selling pressure. In this case, the indicator moves up when the closing price is closer to the highest high and moves down when the closing price is closer to the low. This allows ADI to analyze whether the market is experiencing accumulation, which is buying pressure, or distribution, which is selling pressure. The indicator tends between -1 and 1, where values closer to 1 indicate strong accumulation, and values closer to -1 indicate strong distribution. In this case, no window was used since it is cumulative.

Money Flow Index

The Money Flow Index, or MFI, is an indicator that analyzes the flow of money into and out of an asset, by considering the price movements and trading volume. In this case, the MFI is expressed as a value between 0 and 100. It helps measure the strength of buying and selling pressure over time, providing insights into the market movement and behavior. Unlike the previous volume indicators, MFI was implemented with a window which is:

- MFI 14: This window is a short-term period that captures money flow trends while maintaining a balance between sensitivity and stability.

Chaikin Money Flow

The Chaikin Money Flow is an indicator that monitors and analyzes the accumulation and distribution of stock over a specific time. In this case, the values of this indicator move between -1 and 1, where positive values indicate buying pressure and negative values indicate selling pressure. When the CMF moves right around the zero line, this demonstrates that there is relatively equal buying and selling pressure and no clear trend. The window that was used for this indicator is:

- CMF 21: This is the default CMF period. It allows the indicator to capture medium-term trends in buying and selling activity while smoothing out short-term movements.

Importance of Volume Indicators

Volume indicators are crucial for understanding the strength behind price movements. While prices alone help us analyze the general direction of the market, volume indicators reveal whether trends are supported or not by trading activity. By implementing these indicators, the trading strategy is able to analyze the intensity of buying and selling pressure and validate potential trading signals, improving the efficiency of the predictions.

Combination of Indicators

Using a combination of momentum, volatility, and volume indicators allows this trading strategy to gain a more complete understanding and perspective of market behavior. Momentum indicators help identify the direction and strength of price movements and trends. Volatility indicators reveal how stable or unstable the market is. Lastly, volume indicators measure whether trends in the price are supported by trading activity or not. The combination of these types of indicators allows the strategy to be more reliable and robust.

Normalization

To ensure that the model processes the input features effectively, different normalization and scaling techniques were applied, this depending on the type of indicator and the way it behaves. Since the indicators do not share a common scale, the normalization process allows their values to be comparable and prevents any feature from dominating in the training process due to differences in magnitude. With this said, using the same scaling method for all indicators would not be appropriate since each indicator operates on different numerical scales. Therefore, the normalization that was done was customized based on the characteristics of each indicator.

Fixed Scale Normalization

Some indicators originally operated on a fixed scale ranging from 0 to 100. For these types of indicators, the normalization process consisted of simply dividing their values by 100 so that their results would be within the 0 – 1 range. This ensures that the output given by these indicators is numerically consistent with the other features in the model. In this case, three indicators were normalized with this process and they are:

- Relative Strength Index (windows = 7, 14, 21)
- Stochastic Oscillator (windows = 12, 26, 40)
- Money Flow Index (window = 14)

Normalization Relative to Price and Position

On the other hand, some indicators depend directly on the price of an asset. In this case, to normalize these outputs, they were divided by the current closing price. This transforms the indicators into relative measures, allowing them to be compared correctly with the rest of the indicators. By scaling this way, the indicators directly

reflect their magnitude relative to the current value in the market. This method reduces the impact of large differences between prices of different assets, making the model more stable when comparing. The indicators that were normalized this way are:

- Exponential Moving Average (windows = 10, 21)
- Average True Range (windows = 14)

With this said, for other indicators, instead of dividing by the closing price, the position of the closing price within the range is calculated. This is done by calculating the difference between the closing price and the lower band and then dividing them by the total width of the bands or channel, in other words, divided by the difference between the high and low. This also gives us a value between 0 and 1, which allows these indicators to be compared correctly with the rest. Values close to 0 indicate that the closing price is near the lower bound, while values closer to 1 indicate that this price is close to the upper bound. The two indicators that were normalized this way are:

- Bollinger Bands (windows = 15, 20)
- Donchian Channel (window = 20)

Z-Score Normalization

Some indicators do not necessarily work on a set scale and can take both positive and negative values, depending on the conditions of the market. In other words, these indicators tend to fall between -1 and 1. For these types of indicators, the Z-Score normalization was applied. In this case, the mean is subtracted to the value, and this is then divided by the standard deviation. Values below the mean would be negative, and values above the mean would be positive. This method ensures that these indicators are around 0 and prevents features with larger ranges from dominating the model. Z-Score normalization doesn't necessarily force all values into a 0 to 1 range, but it does bring them into a more common scale. The indicators that were normalized this way are:

- Rate of Change (window = 10, 20, 45)
- On Balance Volume
- Volume Price Trend
- Accumulative Distribution Index

Min-Max Normalization

Lastly, some indicators result in values that can vary a lot depending on the behavior of the market. This means that these do not really have a fixed or set scale. In this case, the Min-Max normalization method was used. With this method, the minimum values of a feature get transformed into 0, while the maximum values transform into 1. The formula is the difference between the value and the minimum divided by the range. Values closer to 0 represent data near the minimum, while values closer to 1 correspond to data near the maximum. The only indicator that was normalized using this method is:

- Chaikin Money Flow (window = 21)

Normalization of Closing Price

The closing price of the asset was also normalized using the Z-Score method. In this case, the mean of the closing price is subtracted from each value, and the result is divided by the standard deviation. This transforms the closing price into a standardized feature centered around 0, where positive values indicate values above the mean and negative values below the mean. This process was also done to the price to prevent any domination from any of the features.

Class Imbalance Strategy: Threshold Adjustment

When doing financial trading, the number of long, short and hold signals tends to be somewhat imbalanced, leading to an inefficient model that does not predict minority classes correctly. To fix this issue, a class imbalance strategy using threshold

adjustment was implemented. Instead of using a default decision threshold, like 0.5, for classification, the threshold was modified to better balance the number of predicted longs, shorts, and holds. In this case, the signals are obtained considering the percentual change in the price of the asset in a 10-day period. This period was chosen because the data is daily, and using a shorter horizon would result in a very small price change that would not be significant for trading decisions. With this said, the threshold cannot be set too high, since drastic price changes are not expected within a short period. It must be consistent with the 10-day horizon to ensure that the signals remain realistic and aligned with typical market movements.

Target Variable Definition

Trading Signals

To be able to train the model to generate effective trading signals, it is necessary to define a target variable, which in this case is the recommended position the trader should take based on the expected movement of the market. With this said, the trading signals are labeled in the following way:

- Long Position (1): A long position should be taken when the price of an asset is expected to increase. In this case, the trader buys the asset anticipating that they will be able to sell it later at a higher price, obtaining a profit from the difference. When the model indicates a long signal with the number 1, it implies that the projected price movement is positive and its tendency justifies taking a buying position.
- Short Position (2): A short position should be taken when the price of an asset is expected to decrease. The trader sells the asset with the intention of buying it in the future at a lower price, gaining a profit with the difference in prices. When the model's output is a short signal represented by the number 2, it indicates that the price is expected to decrease significantly in the future.

- Hold (0): A hold signal is assigned by the model when the expected movement in the price is not as significant or large to open or close a position. When the model signals 0, it avoids trades during periods of minimal price fluctuations, reducing unnecessary exposure to market noise.

Threshold Logic for Class Assignment

For this project, a threshold was implemented to be able to define the minimum relative change in the price that is required to generate a long, short or hold signal. In the case of this trading strategy, a threshold of 0.014, or 1.4%, was used. The target signal for a given day is determined by comparing the closing price 10 days ahead to the current closing price. The threshold is used in the following way:

- If the future price is at least 1.4% higher than the current price, a long position is assigned.
- If the future price is at least 1.4% lower than the current price, a short position is assigned.
- If the future change in the price is minimum and less than 1.4% in either direction, a hold signal is assigned, indicating that no significant movement is expected.

The established threshold ensures that only meaningful price movements are considered, avoiding noise from small and insignificant fluctuation in the price. The choice of 0.014 as a threshold is consistent with the 10-day period. It is a large enough value to represent a relative price change but not so large that too few signals are generated.

Distribution of Signals in Training Data

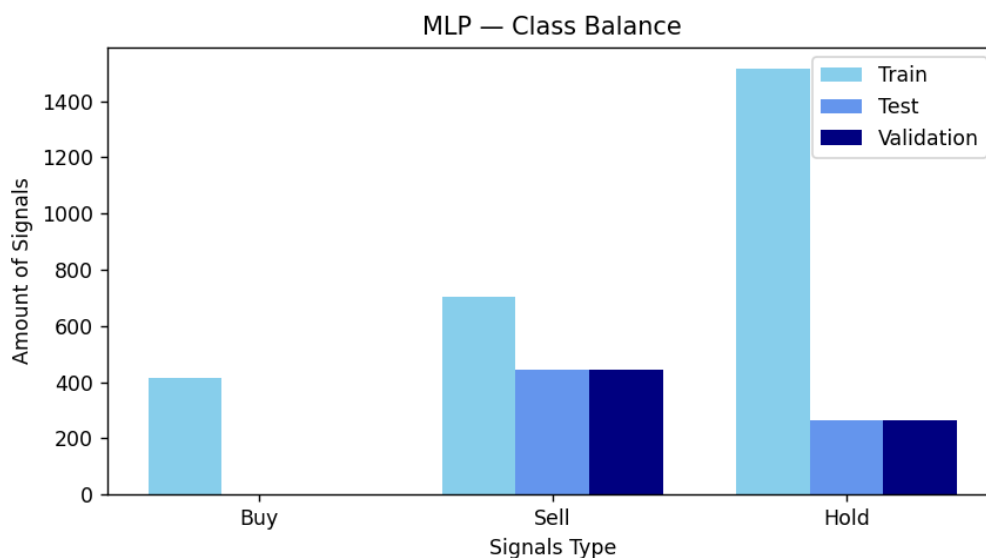
After applying the threshold logic, the trading signals obtained result in a natural imbalance. Typically, hold signals are much more frequent due to most price movements over the 10-day horizon not extending the threshold established. With

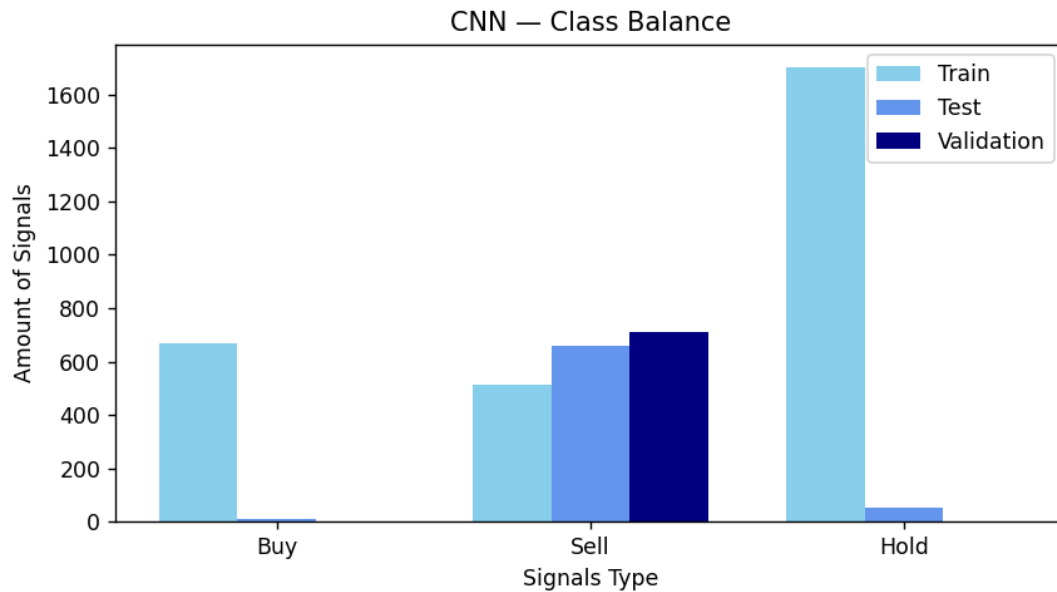
this said, long and short positions are less frequent, reflecting periods with more significant changes in price.

This class imbalance is important to take into consideration because models trained without caring for it may become biased toward predicting the majority class, which in this case would be the hold position. This would reduce the performance for minority classes, like long and short positions. This can be seen in the next results of both the MLP and CNN models implemented in this strategy:

TRADE STATISTICS: MLP	BUYS	SELLS	HOLDS	TOTAL TRADES
TRAIN DATA	413	703	1,516	1,116
TEST DATA	0	444	265	444
VALIDATION DATA	0	445	266	445

TRADE STATISTICS: CNN	BUYS	SELLS	HOLDS	TOTAL TRADES
TRAIN DATA	689	515	1,704	1,204
TEST DATA	8	658	51	666
VALIDATION DATA	0	711	0	711

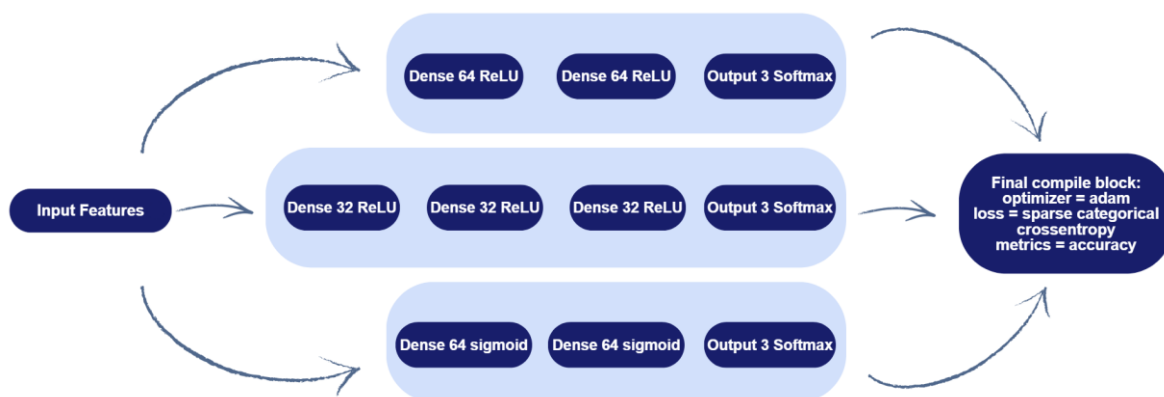




With the previous data, it is very evident that there is an imbalance in the classes in both neural networks. Also, in the train data of both models, the hold positions dominate the rest of the data, this is due to the higher frequency that this occurs. This training data strongly affects how the test and validation will be done. For this reason, a class imbalance strategy that was mentioned previously was implemented.

Multi-Layer Perceptron

Architecture



Layer Description

As mentioned previously, the Multi-Layer Perceptron model consists of three main layers which are the input layer, hidden layers, and output layers. These are described as follows:

- **Input Layer:** The input layer receives all the features from the dataset, which in this case is the preprocessed financial data for the asset NETFLIX. These features include normalized momentum, volatility, and volume indicators, as well as the historical closing prices.
- **Hidden Layers:** The hidden layers consist of fully connected neurons that process the input features to be able to learn complex relationships. Each layer applies an activation function which allows the neural network to model interactions between indicators. The number of hidden layers and neurons per layer can be configured specifically to capture market patterns from the training data. The processing in the hidden layers is iterative and sequential. Each layer receives the output of the previous layer, applies its parameters, and passes the result to the next layer. This allows the neural network to gradually transform the features into more complex information to predict trading signals.
- **Output Layer:** The output layer consists of three neurons that correspond to the three possible trading signals, which are long, short, and hold. This is done with another activation function which transforms the final values of the neurons into probabilities to be able to determine which trading signal is most probable.

Parameters

The configuration of this neural network involves several key parameters that allow the model to learn patterns from historical data. With this said, the parameters that were considered in this architecture are the following:

- **Number of Hidden Layers:** In this case, the number of hidden layers that were taken into consideration in the hyperparameter search space were 2 and 3 layers. This slight variation in the number of hidden layers allows the model to compare how increasing this can affect the model's performance capacity. A greater number of hidden layers usually enables the network to capture more complex relationships in data, although there is also a risk of overfitting.
- **Dense Units:** Each hidden layer consists of a fixed number of neurons. In the case of this strategy, 32, 64, 128, and 256 neurons were used in the search space. This determines how much information the model can process in each stage. Normally, more neurons result in a richer analysis of the input data.
- **Activation Functions:** The hidden layers of this model make use of the Rectified Linear Unit (ReLU), Sigmoid and ELU activation function. ReLU introduces non-linearity and allows the model to learn more complex patterns in the data, while Sigmoid maps the outputs into a value between 0 and 1. Also, ELU is an advanced version of the ReLU activation function. ELU allows the model to receive negative outputs, which improves the robustness of the model. On the other hand, the output layer uses the Softmax activation function, which converts the final outputs into normalized probabilities for each class.
- **Optimizer:** The optimizer that was used in this strategy was Adam. This optimizer adapts learning rates during training for each parameter. This allows the model to work with volatile and non-linear information more effectively, especially when working with stock price movements.
- **Loss Function:** The model uses Sparse Categorical Crossentropy as the loss function. Since the output consists of three distinct categories, this loss function is appropriate because it measures how well the predicted probabilities are in comparison to the true trading signals.
- **Batch Size and Epochs:** The model was trained using a batch size of 32, which means that the model processes the data in groups of 32 samples at a time before updating the weights. Additionally, the model was trained for 100

epochs, meaning that the entire dataset was passed through the network multiple times to refine the learning process.

Convolutional Neural Network

Convolutional Structure

The CNN model used in this trading strategy is designed to process sequential stock prices and extract useful temporal patterns. The structure begins with an input layer which receives the time series as a one-dimensional (1D) signal. After this, the neural network incorporates different convolutional blocks, where each one consists of a 1D convolutional layer followed by a MaxPooling layer.

The convolutional one-dimensional layers apply learnable filters to the input data to be able to capture short-term movements in price dynamics. In this case, each convolutional block uses filters with a specific kernel size, which means that the model analyzes patterns across windows at a fixed number of steps at a time. Additionally, causal padding is implemented into the model, ensuring that the model does not use future information when making predictions, preventing look-ahead bias.

After each convolutional layer, a MaxPooling layer is applied. This step is used to reduce the dimensions of the feature maps, which reduces the number of parameters used to learn. This helps the neural network retain only the most relevant features. With this said, the number of filters increase gradually with each layer, which allows the model to learn more complex patterns and trends in the movement of the stock prices.

Once all the convolutional layers have finished extracting important data and features, the result or output is flattened, which converts the multidimensional output into a one-dimensional result. This flattened output is then passed to a dense layer, where there is a defined number of neurons that controls how much information the

model processes during this stage. Then, an activation function is applied to introduce non-linearity to the model.

The final layer is a dense output layer with three neurons that represent the three possible trading signals of this project. Here, the SoftMax activation function converts these outputs into probabilities. The model is then compiled using the Adam optimizer to be able to adapt learning rates during the training of each parameter. Also, the Sparse Categorical Crossentropy loss function is used to measure the efficiency of the trading signals obtained.

Kernel Size

In the convolutional layers of this CNN model, which were explained previously, a kernel size of 5 was used. The kernel size determines the number of steps that the convolutional filter analyzes at once. This allows the neural network to capture short-term patterns and trends in stock price fluctuations. The kernel size of 5 was chosen because it provides a balanced window. It is large enough to detect meaningful short-term movements, but not so large that it smooths over important variations in the data.

Pooling Strategy

After each convolutional layer, a MaxPooling layer is applied. The purpose of implementing this type of strategy is to reduce the dimensionality of the feature maps produced in the convolutional filters. By only keeping the maximum value within each pooling window, the model keeps only the most relevant features, discarding less significant information. This reduction helps the model with preventing overfitting and allows the neural network to focus on the most important patterns in the data. In this case, the pooling size that was used was 2. This means that for every two values in the feature map, only one is selected, which would be the highest value.

Parameters

The CNN model used in this trading strategy incorporates several parameters that control how the network learns and processes the information. With this said, the parameters that were implemented into this model are the following:

- **Number of Convolutional Layers:** This parameter determines how many convolutional blocks are stacked. By increasing the number of layers, the model is able to learn more complex patterns, but this would also mean a higher risk of overfitting. Due to this, the model was done with a hyperparameter search space of only 3 convolutional layers.
- **Number of Filters:** This parameter specifies how many filters are applied in the first convolutional layer. As mentioned previously, after each convolutional block, the number of filters is doubled. This is done with the intention of the initial layers capturing basic patterns, while deeper layers learn more detailed and complex patterns. In this case, the search space was 64 and 96.
- **Activation Function:** Different activation functions were tested with this model, including ReLU and Sigmoid. These two activation functions were mentioned previously in the MLP model. On the other hand, the Softmax activation function was also used on the output layer.
- **Dense Units:** After the convolutional layers, the model includes a dense layer. The number of neurons in this layer controls how much information the model retains from the extracted features before making decisions. In this case, two values were considered in the search space, 96 and 128 units.
- **Optimizer, Loss Function, Epochs, and Batch Size:** Lastly, the optimizer, loss function, epochs and batch size used in this CNN model are the same as the ones used in the MLP model. With this said, the optimizer used was Adam, the loss function implemented was Sparse Categorical Crossentropy, the epochs used were 100, and the batch size considered was 32.

Training Specifications

As mentioned previously, the same training specifications were used for both neural networks. Although each model has different parameters that can vary, these training conditions remain constant to isolate the effect of parameter changes on performance. The optimizer used for these models is Adam. This optimizer has the function of adjusting the weights of the model during the training period. Adam combines two different methods (momentum and RMSProp), which help the model adapt to changes and optimize successfully.

In terms of the loss function, the one used in this neural network is Sparse Categorical Crossentropy. This loss function is helpful because it measures the difference between the model's predicted probability distribution and the true class. It is used when the problem has more than two output classes. This function works with the next formula:

$$L = -\log(p_y)$$

As seen previously, Adam is calculated by taking the negative logarithm of the predicted probability of the correct class (p_y). This model aims to maximize the probability of correct prediction and minimize the loss itself.

The other specifications that were used during the training period were a batch size of 32 and 100 epochs. Each model was trained under these conditions until the best-performing version was identified.

Class Weighting

As mentioned previously, class balance was achieved by adjusting the threshold parameter, referred to as alpha (α), which determines the percentage change required to generate trading signals. In this study, alpha values were tested within a range between 0.05 and 0.2, corresponding to typical ranges used in trading. During experimentation, it was observed that when alpha was too high, the models achieved good validation accuracy but suffered from class imbalance, generating very few buy

and sell signals. Conversely, when alpha was close to 0, the class balance improved, but the model tended to produce false signals, leading to poor trading performance. After several trials, the alpha value that provided the best overall results and maintained an appropriate class balance was 0.014.

Experiment Tracking

Experiment Setup and Logging Structure

To track every trial of the CNN and MLP models, the tool used in this project was MLFlow. This platform allows the user to save, organize, and compare different experiments through performance metrics, making it easier to determine which configuration performs best.

The workflow begins by training the models in Visual Studio Code. Once the training is completed, the user accesses MLFlow, where each experiment is stored and grouped based on the type of neural network. Every experiment contains recorded metrics that indicate the quality of the model's performance. When a model achieves good results, it is saved as a versioned model within MLFlow, allowing it to be tracked and retrieved later.

Finally, the selected model version is loaded back into Visual Studio Code to perform backtesting and generate predictions. This step allows the model's trading performance to be evaluated using additional financial and performance metrics.

Parameters Tracked for each Model

For both neural networks, it was necessary to determine which parameters resulted in the best performance for each experiment. For both MLP and CNN models, the parameters tracked were the number of dense units, convolutional layers, and the activation functions. Additionally, the number of convolutional layers was also

tracked for the CNN models. After each experiment, the best performing results were saved. This approach helped identify trends among the most effective models. When such trends were observed, other suggested parameters were modified to align with those of the best model, facilitating the obtention of the best model.

Metrics Recorded

Accuracy

Accuracy measures the percentage of correct predictions made by the model during the training period. It indicates how effectively the neural network is learning from the training data. It is important to monitor this metric alongside test accuracy because if training accuracy is significantly higher than the test accuracy, the model may be overfitting.

Validation Accuracy

Validation accuracy also measures the percentage of correct predictions, like the accuracy mentioned previously, but it is evaluated on data that the model has not seen before, which is the validation data. A high validation accuracy indicates that the model generalizes well to new data. As mentioned earlier, a large gap between training and validation accuracy signals potential overfitting.

Validation Loss

Similar to validation accuracy, validation loss helps evaluate how well the model performs on data it has not seen during training. It is important to achieve a low validation loss value, as this indicates that the model is effectively learning market patterns. On the other hand, a high validation loss suggests overfitting, meaning the model has primarily learned from past data without generalizing well to new situations.

F1 Score

The F1 Score evaluates the balance between precision, which is how many of the predicted positive cases were correct, and recall, which is how many of the actual positive cases were successfully identified. This metric is particularly useful when working with imbalanced classes, as it provides a more reliable measure of model performance beyond accuracy alone.

Test Performance

Test performance is not a metric, but rather the final evaluation phase in which the model predicts unseen data. This step simulates real-world trading conditions, where future market behavior is unknown. If the model performs similarly in both the validation and test phases, it suggests stable and reliable performance. However, if test performance declines significantly compared to validation, the model may not generalize well and could still be overfitting.

Model Comparison: MLP vs. CNN

PARAMS AND METRICS	MLP	CNN
LAYERS	3	3
FILTERS	DOES NOT APPLY	32
DENSE UNITS	32	64
ACTIVATION	RELU	SIGMOID
ACCURACY	0.6871	0.5189
VAL ACCURACY	0.4090	0.4499
VAL LOSS	2.2290	1.015
F1 SCORE	0.2988	0.39

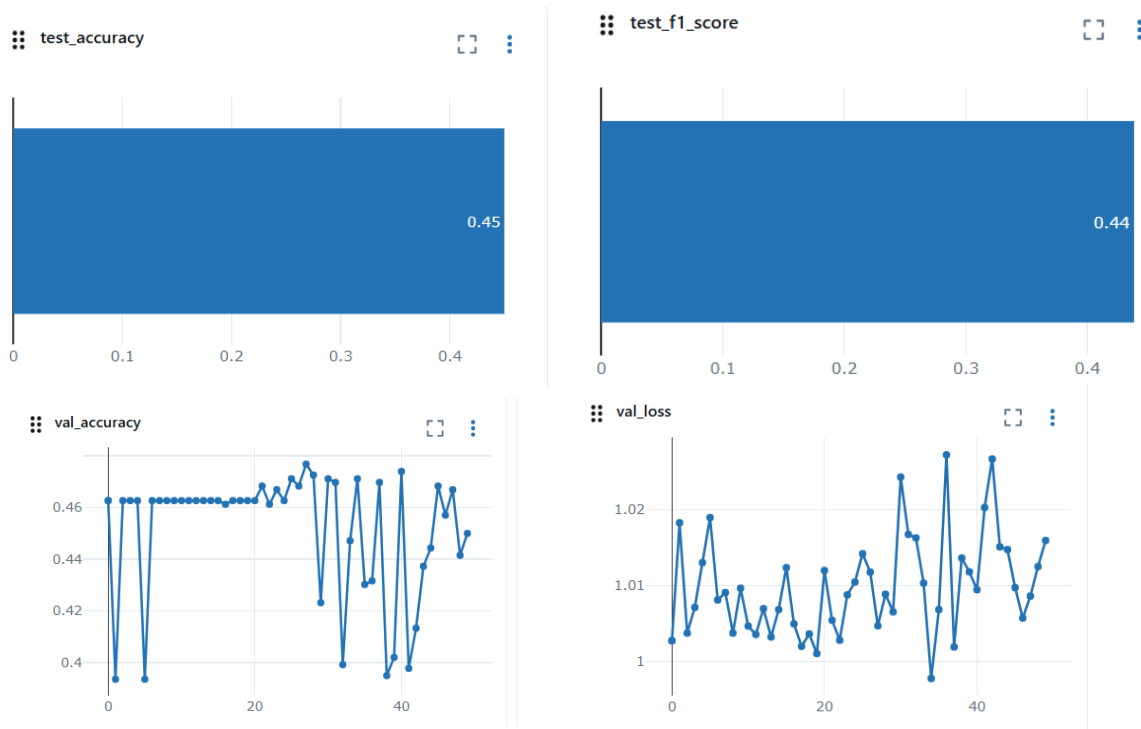
With this table, it is easy to see that the main differences between the best version of each type of neural network lie in the number of layers, the number of dense units, and the activation functions, which vary across models. On the other hand, it is also important to mention the implementation of filters in the CNN model, making a slight difference in results. With this, it is noticeable that CNN model is a better option, because even though their validation accuracy is similar, the accuracy in the MLP is greater than in CNN, indicating that it may be slightly overfitting, complemented by a high validation loss. Also, the F1 Score determines that the CNN model has a better performance in predicting signals.

Selected Model

When comparing the results of both models, the CNN model was chosen. Although the MLP has a higher validation accuracy, it exhibits a larger gap between accuracy and validation accuracy, and its validation loss is higher, indicating possible

overfitting. On the other hand, the CNN- model has a slightly lower validation accuracy, but it is closer to the training accuracy, and its validation loss is near 1, demonstrating a better balance and reducing the likelihood of overfitting. For these reasons, the CNN model was selected to predict the trading signals.

Once the model was selected, with the use of graphs and metrics it is possible to determine the efficiency of the test performance. With this said, this can be appreciated in the following visuals:



The previous graphs display the values obtained in the test performance, where the test accuracy and F1 score are similar. Although these values could improve, they do indicate that the model is balanced and not overfitting. Additionally, the validation accuracy and validation loss graphs indicate how these metrics evolved over time, showing all the highs and lows in the model's performance until reaching the values reported in the test performance.

Data Drift Monitoring

Drift Monitoring Methodology

Loading Data and Define Windows

To begin the process, 15 years of historical price data for the asset are obtained. The dataset is then divided into train, test, and validation segments. After the split, technical indicators are added to each subset, and the data is normalized to ensure consistent scaling across all periods.

Once the dataset is ready, the window size and steps are defined. These parameters determine how often the distributions will be compared over time. For example, this can be every n days or after a specified number of observations, making it possible to detect gradual or sudden changes in the underlying data structure.

K-Test and P-Values

In this step, the training data is compared to the data in each defined window. To do this, the Kolmogorov-Smirnov (KS) test is applied to both distributions, which evaluates how different they are from each other. Each time the test is executed, it returns two values:

- KS Statistic (KS_stat): indicates the magnitude of the difference between the two distributions.
- P-value: represents the statistical significance of that difference.

If the p-value falls below a selected confidence threshold, it suggests that the feature's distribution has changed over time, which means that data drift has occurred for that variable.

Storage and Data Visualization

The values obtained from the KS statistics and the p-values are stored in separate dataframes. This allows the generation of different visualizations that help interpret the presence and behavior of data drift over time.

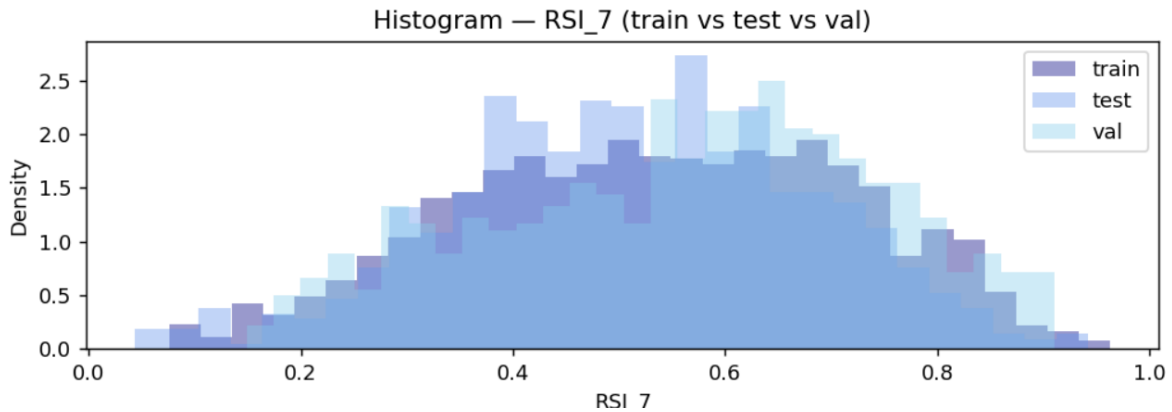
In this case, histograms can be used to compare the distribution of each feature in the train, test, and validation sets. If the shapes of the histograms differ noticeably, it suggests that the feature may have shifted. This is a simple way to visually detect drift in the data.

Additionally, p-value plots also provide insight into when drift occurs. These plots show moments where the p-value is high or when it drops below the confidence threshold, indicating the specific point in time and market conditions where a feature's distribution changed.

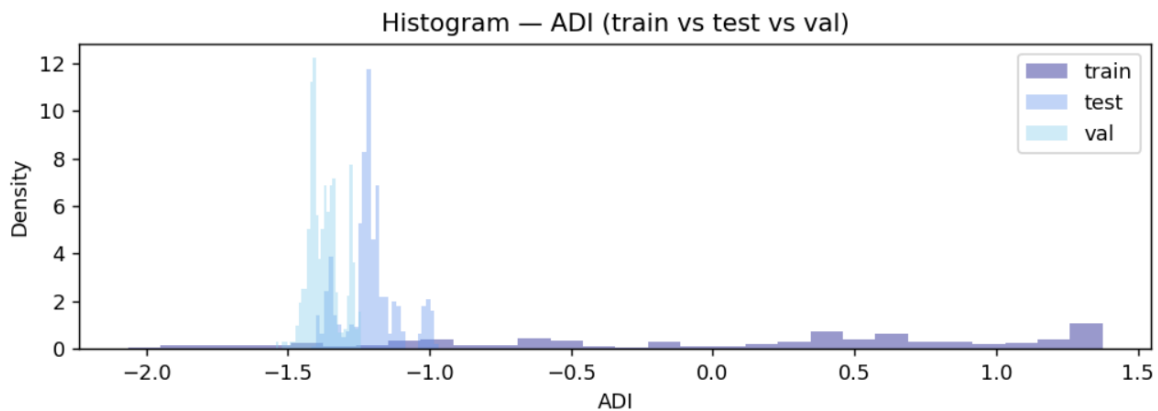
By analyzing the stored dataframes, it is also possible to identify which features experience drift most frequently. This information is useful for deciding which features are reliable for the model and which ones may need to be replaced, removed, or adjusted.

Drift Dashboard

For a clearer visualization of the distribution of each feature across the different splits, histograms were created. This allows for an easier observation of whether the distribution remains consistent across periods or changes, clearly indicating any drift in the feature.



In the previous histogram, it is possible to observe the distributions of the Relative Strength Index with a window of 7 across the periods of train, test, and validation. This graph indicates that there is no evident drift, as the three distributions have nearly the same shape, their centers remain within the same value range, and none appear to be skewed to either side.



Unlike the RSI histogram, the histogram for the Accumulative Distribution Index feature shows significant drift. The shapes of the three distributions differ, none share the same center, and the density in the training set is lower than in the test and validation sets. While histograms do not indicate when the drift occurs, they provide a simple way to visualize and understand the magnitude of the drift.

Features with Drift

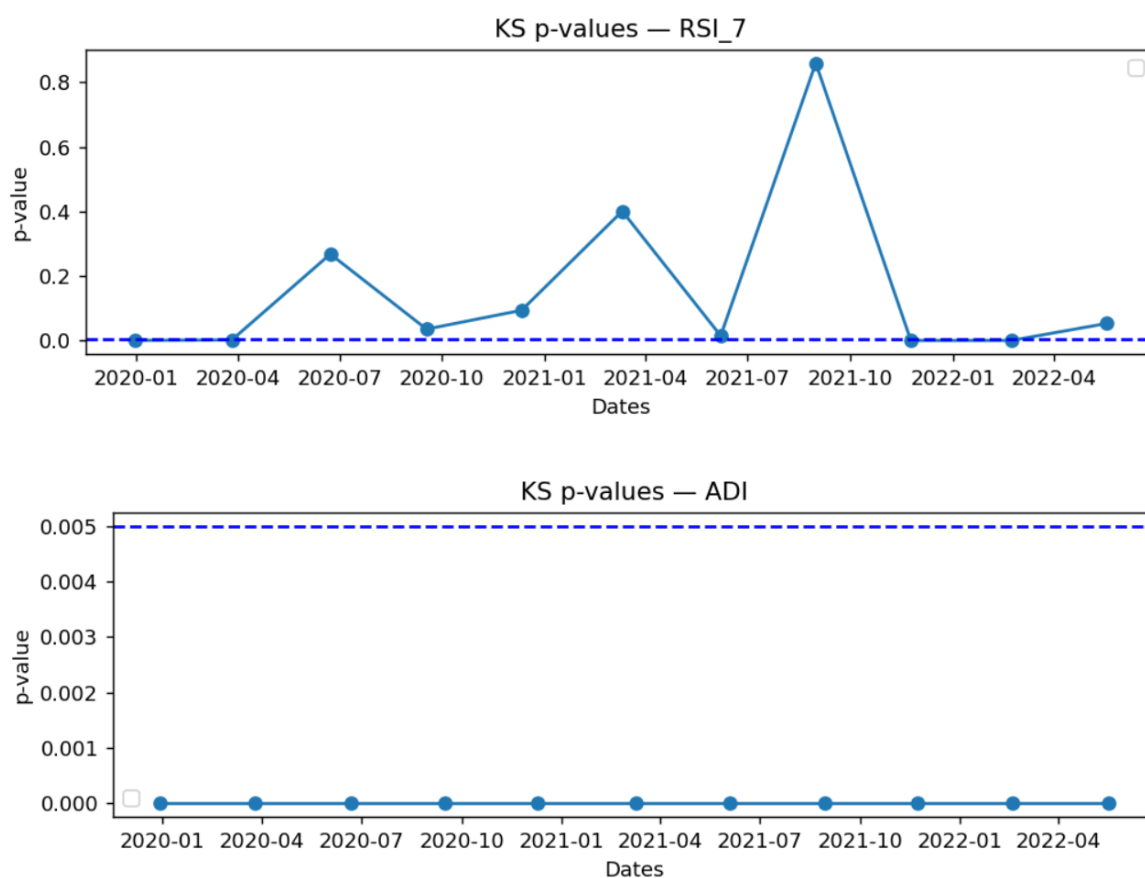
Histograms made it easy to determine whether drift occurred, but they do not explain why it happened in a particular feature. The following table displays each feature, its KS p-value, and whether it experienced drift during the period. If the p-value is below the confidence level, which is $\alpha = 0.05$, the feature is considered to have experienced drift.

Feature	Test		Val	
	p_value	Drift	p_value	Drift
ADI	0.000000	Yes	0.000000	Yes
ATR_14	0.000351	Yes	0.000185	Yes
BB_pos_15	0.343241	No	0.152751	No
BB_pos_20	0.235445	No	0.098526	No
CMF_21	0.040990	Yes	0.015255	Yes
DON_pos_20	0.159850	No	0.096398	No
EMA_10	0.162152	No	0.188694	No
EMA_21	0.048725	Yes	0.113142	No
MFI_14	0.098686	No	0.093426	No
OBV	0.000000	Yes	0.000000	Yes
ROC_10	0.098978	No	0.125261	No
ROC_20	0.028821	Yes	0.096324	No
ROC_45	0.000544	Yes	0.013356	Yes
RSI_14	0.003472	Yes	0.045951	Yes
RSI_21	0.000144	Yes	0.019536	Yes
RSI_7	0.156886	No	0.148564	No
Stoch_12	0.355998	No	0.172340	No
Stoch_26	0.069541	No	0.137030	No
Stoch_40	0.035406	Yes	0.051582	No
VPT	0.000000	Yes	0.000000	Yes

The analysis of this table reveals whether each feature experienced drift during the test and validation periods. It is evident that most features show some degree of drift, with some being more pronounced than others. This information is useful for identifying features that do not follow the training distributions and supports decisions regarding whether to modify or replace those features.

Timeline of Drift Periods

One of the most important graphs for understanding p-values and identifying when drift occurs is the p-value timeline. Sometimes it is not enough to know that a feature has drifted, it is also necessary to know when it happens, and the following graph provides that information. By plotting the p-values over time along with the dates on which the statistical tests were performed, it helps determine when the feature experienced drift and understand the market context in which it occurred.



The following graphs indicate that RSI did not exhibit drift during 2020, while ADI experienced consistent drift. The dates are important, as they allow us to understand that the differences in p-values are related to the pandemic crisis. During the pandemic, stock prices suffered historic declines, resulting in extreme volatility and abrupt changes in volume. For this reason, the RSI maintained p-values not on 0, while ADI did not. The RSI is a relative strength indicator, so its normalization is fast,

and it adapts well to changes. With this said, strong market movements do not significantly affect the feature. In contrast, ADI combines both price and volume, so constant changes in these factors impact its performance. The timeline not only shows how high or low the p-values are but also reveals when these changes occurred and the global context surrounding them.

Backtesting Methodology

Signal Generation

For the signal generation and buy/sell process, the following procedure was implemented. First, the split dataset is passed through a function that evaluates the percentage change in the closing price according to the alpha condition. If this change exceeds the defined threshold, the function adds a new column to the dataset containing three possible signals, which are 1 = buy, 2 = sell, and 3 = hold.

This enriched dataset is then used in the backtesting process. During backtesting, each row is read sequentially to identify whether a trading signal is present. When a signal is detected, positions are either opened or closed accordingly, and each transaction is recorded in a list. Throughout each iteration, as trades are executed, the portfolio value is continuously updated. Once the dataset is fully processed and no additional trading signals remain, all open positions are closed, and the final portfolio value is calculated.

Position Sizing and Management Rules

For position sizing, a fixed size of 100 shares per trade was used for both long and short positions. Each transaction made had a transaction cost of 0.125%, while short positions also included a daily borrow rate equivalent to 0.25% annually.

Long trades were opened only if there was sufficient available cash to cover both the position and its associated commission. Similarly, short trades required enough cash

to cover the entry commissions. To manage risk effectively, the backtest implemented a take-profit limit of 13% and a stop-loss limit of 6% for each position.

At the end of the process, key metrics such as cash value, portfolio value, number of trades, and win rate were evaluated to determine whether the strategy was successful.

Data Drift

Within the backtest, a data drift monitoring procedure was also implemented. This process detects changes in the statistical distribution of the model's features over time. It is performed every 60 steps by comparing a 90-bar rolling window from the test dataset with the training data. For each feature, the Kolmogorov-Smirnov p-values are calculated, indicating whether the distributions in a given period have changed, which would signal the presence of drift. This procedure helps to understand in real time when the model is operating with distributions different from those seen during training, providing the opportunity to rebalance and retrain the model.

Assumptions and Limitations

In the backtesting process, several key assumptions were made to ensure consistency and clarity in the results. One assumption is that commission costs and borrow costs remain constant over time. Another is that the backtest is designed for a single asset, as all parameters and logic are focused exclusively on that asset. Additionally, there is no restriction on the number of simultaneous positions, so as long as there is sufficient cash available, new positions can continue to open. The stop-loss and take-profit levels are fixed and remain unchanged throughout the entire backtest.

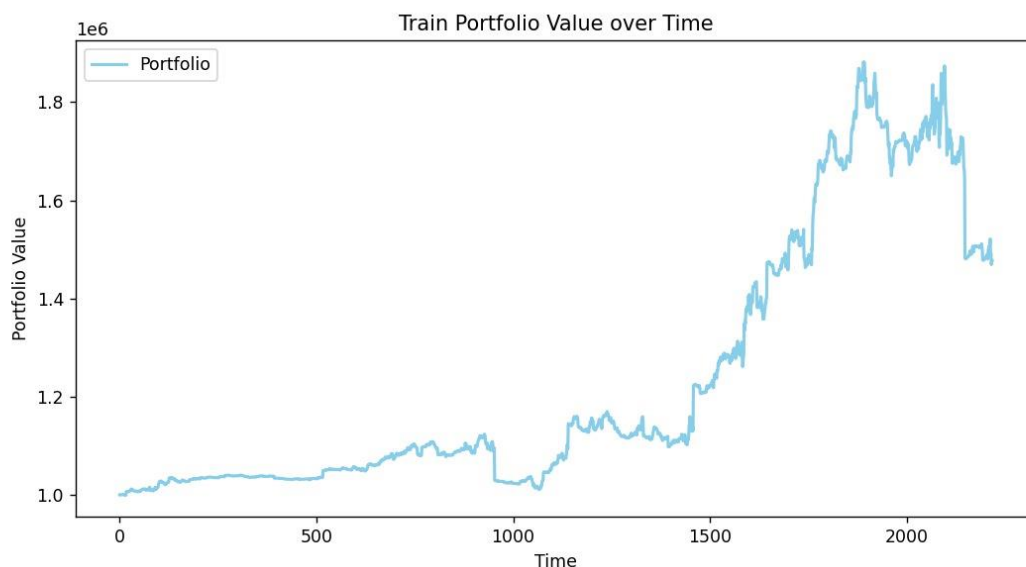
However, the methodology also presents important limitations. There is no margin account or margin call, meaning the strategy could fail to open new positions to cover

losses if available cash runs out. Moreover, commissions vary across countries and brokers, so assuming they are constant does not accurately reflect real trading conditions. Since all parameters remain static, the backtest carries certain risks, particularly regarding the number of shares and the number of shares traded per position. Understanding these assumptions and limitations is essential to interpret the results correctly and to identify the specific scenarios where the backtest is applicable.

Best Results and Performance Analysis

Equity Curve Plots

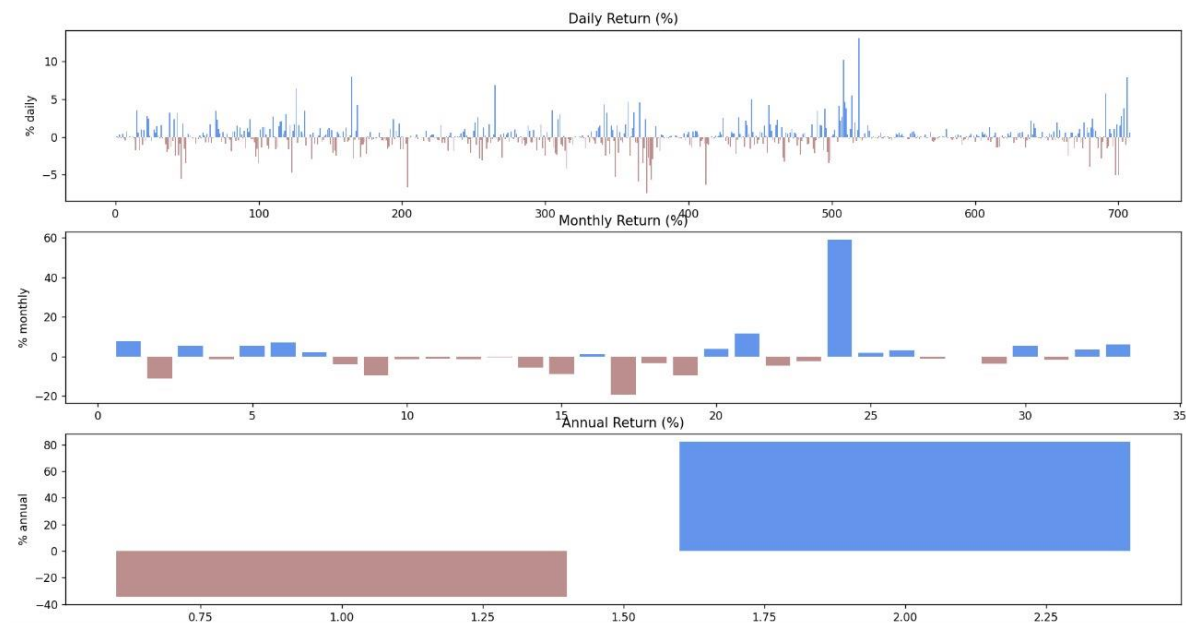
Train Period



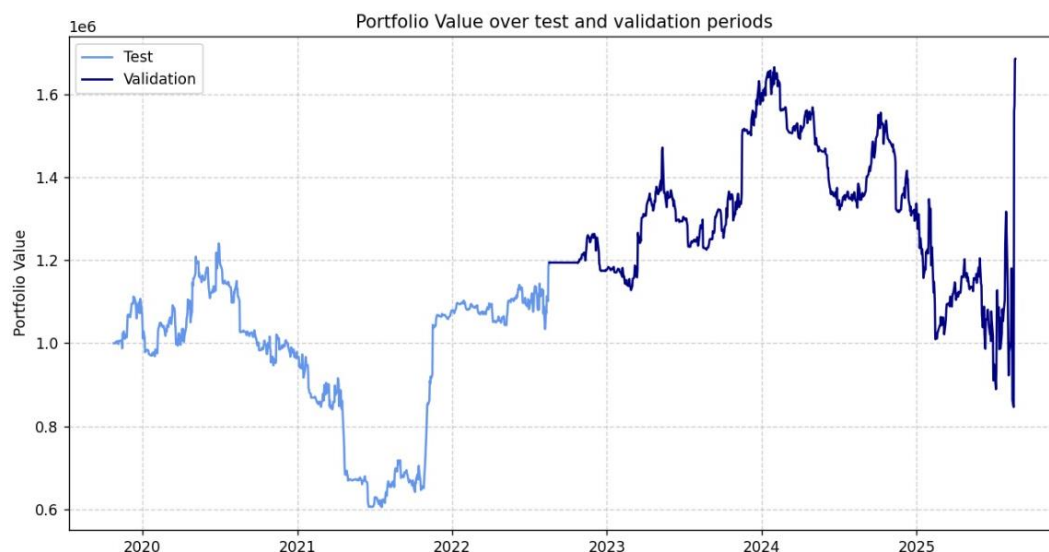
In the previous graph, the evolution of the portfolio value during the training period can be observed. It is evident that there is an upward trend, resulting in a positive outcome with decent gains. This indicates that the model was able to learn from the training data effectively, capturing profitable patterns in the stock price movements. On the other hand, the portfolio demonstrates typical fluctuations due to market

volatility, reflecting the model's capacity to also adapt short-term market movements, preventing an overly smooth result. It is important to mention that, while the training results are promising, an evaluation of the model on unseen data is necessary to confirm its generalization.

In the following graph, the daily, monthly and annual returns of the training data can be observed, providing a comprehensive view of the portfolio's performance over different time frames. In the first section, the daily returns show evident volatility, however, this can be considered balanced. In other words, the amount of positive and negative returns is similar. With this said, there is a noticeable peak in the positive returns, indicating a slight dominance of gains over losses. In the monthly returns, a similar behavior can be observed. In this case, there is also certain balance between positive and negative returns and even showing the same peak in the positive returns. In terms of the annual returns, the first year the general annual return was negative, however, in the following year, this is compensated as the annual return is positive and even greater than that of the previous year.

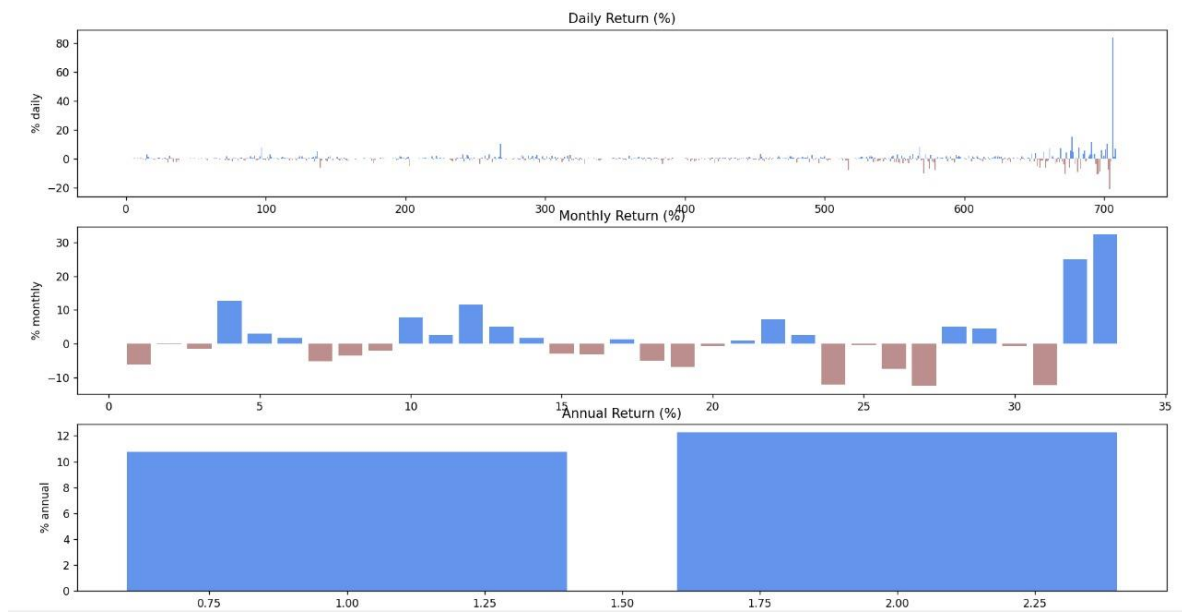


Test and Validation Periods



The previous graph demonstrates the evolution of the portfolio value on both the test and validation periods. The test line, shown in a light blue color, evidently shows that there was a slight downfall in the stock price, but despite this decline, the portfolio maintains an upward tendency when reaching the validation period shown in the dark blue line. This indicates that the model was able to recover from relatively short-term losses. On the other hand, when comparing this graph with the one made with the training period, we can see that, during the test and validation periods, the model reflects a greater amount of volatility, which is expected given that the model is now predicting with unseen data. These fluctuations demonstrate the undeniable risk that there is when working with financial markets.

In the following graph, the daily, monthly and annual returns of the test and validation data can be observed. Unlike in the training data, there is a greater dominance of the positive returns in all three time frames. This dominance is very evident at the end of both the daily and monthly periods, where there is a significant increase in the positive returns, which is beneficial as it indicates portfolio gains. In terms of the annual returns, in both years the returns are positives, demonstrating that the strategy was able to generate consistent gain over the long term, recovering from any short-term losses and producing an overall profitable outcome for the portfolio.



Numerical Results

PERIOD	PROFIT	RETURNS
TRAIN	\$1,477,014.15	47.70%
TEST	\$1,194,092.07	19.40%
VALIDATION	\$1,630,750.09	36.56%
TEST / VALIDATION	\$1,630,750.09	63.07%

In the previous table, the profits and returns of the trading strategy are mentioned. In the train period, the profit is \$1,477,014.15 with returns of 47.70%. With the testing data, the profit is slightly lower with a value of \$1,194,092.07 and returns of 19.40%. In terms of the validation period, the profit was the greatest of the three periods with a value of \$1,630,750.09 and returns of 30.56%. This demonstrates that the trading strategy was able to generate consistent gain across all periods, with the validation period showing the highest profit. Although the returns vary between periods, the strategy maintains a positive performance throughout, indicating that it is robust and

capable of adapting to different market conditions and capturing trends of unseen data.

Performance Metrics

Sharpe Ratio

The Sharpe Ratio is a performance metric that compares the investment's return and profitability with its risk. In this case, a high Sharpe Ratio indicates that the returns are consistent when comparing it to the volatility or risk that is assumed. On the other hand, if the Sharpe Ratio is a low value, it indicates that the returns do not compensate for the risk that is taken when executing the trading strategy. For this reason, the objective is to obtain a positive Sharpe Ratio, as this would demonstrate that the strategy is balanced with returns that outweigh the risk level taken. With this said, the formula for this performance metric is the following:

$$\text{Sharpe Ratio} = \frac{R_p}{\sigma_p}$$

R_p : Average portfolio return

σ_p : Standard deviation of the portfolio returns (level of risk)

Sortino Ratio

Sortino Ratio is a performance metric that is very similar to Sharpe Ratio, but in this case, this metric only considers the downside volatility that is associated. This metric evaluated how well the strategy compensates for the negative returns instead of the general fluctuations of the asset. In this case, a higher value of this metric is better, and it indicates that the investment generates a good profit considering the downside risk. A negative value would mean that the strategy is not efficient since it does not effectively compensate for the risk taken. The formula for this metric is the following:

$$\text{Sortino Ratio} = \frac{R_p}{\sigma_d}$$

R_p : Average portfolio return

σ_d : Standard deviation of the negative portfolio returns (downside deviation)

Maximum Drawdown

The Maximum Drawdown represents the largest observed drop from a portfolio's peak value to its lowest point before a new high point is achieved. In other words, this metric measures the biggest possible loss that can occur during certain period. In this case, the objective is to obtain a lower value, since this would indicate that the strategy is more stable and exposed to less severe losses. A larger value would mean a higher potential risk. The ideal range for the result of this metric would be lower than 10%, but a value between 10% - 25% is still acceptable. The formula for this performance metric is the following:

$$\text{Maximum Drawdown} = \max \left(\frac{\text{Peak Value} - \text{Through Value}}{\text{Peak Value}} \right) * 100$$

Peak Value: Highest portfolio value before the decline

Through Value: Lowest portfolio value during the decline

This performance metric is multiplied by 100 because this value should be a percentage.

Calmar Ratio

Calmar Ratio is a performance metric that compares the average annual return of the portfolio to its maximum drawdown, which was explained previously. In other words, this metric shows how effectively the trading strategy obtains consistent profits without having large drawdowns, reflecting better risk management. Unlike the Maximum Drawdown, in this case, a larger value above 1 is expected, since this

would indicate a good balance between return and risk of drawdowns. The formula for this metric is the following:

$$\text{Calmar Ratio} = \frac{R_p}{MDD}$$

Average Annual Return: Average annual portfolio return

MDD: Maximum Drawdown

Metric Values

METRICS	SHARPE RATIO	SORTINO RATIO	MAX DRAWDOWN	CALMAR RATIO
VALUES	0.38	0.69	51.20%	0.35

With the previous table, we can clearly analyze the values obtained from the performance metrics. These metrics indicate a moderately performing trading strategy, it definitely is not the best strategy, but it does generate positive returns. The Sharpe Ratio of 0.38 shows that the returns are above the level of risk, but they are not highly efficient in terms of the overall volatility. The Sortino Ratio of 0.69 suggests that the downside risk is managed in an adequate way, meaning that the losses are somewhat controlled. However, the maximum drawdown of 51.20% indicates that the portfolio experienced a significant drop from its peak, reflecting a high exposure to potential losses. Finally, the Calmar Ratio of 0.35 demonstrates that the returns are not the best when comparing them to the maximum drawdown. With all this mentioned, it is clear that the strategy is profitable, but there is definitely room for improvement in risk management and overall stability of the trading strategy.

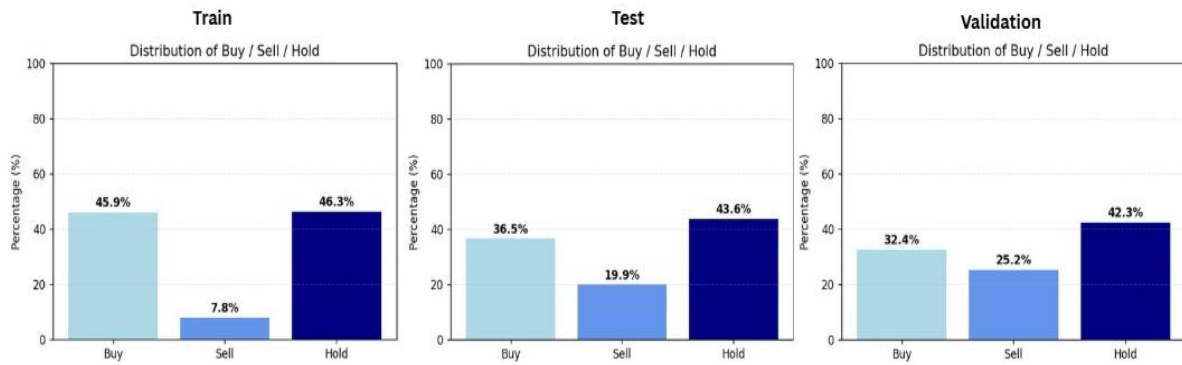
Trade Statistics

The following chart shows the distribution of trades in each period. It is important to note that in the training set there is an imbalance between classes, with one category being the smallest of the three periods. However, in the test and validation sets, thanks to the chosen alpha, the class balance is improved, demonstrating that the class balancing strategy works effectively.

Since the data split was 60% for training, 20% for testing, and 20% for validation, the training set contains the largest number of trades compared to the test and validation sets. It is also important to observe the win rates, as this metric indicates whether the trades were profitable or not. During the training period, the highest win rate appears at 49.41%. This rate decreases to 38.73% in the test period and then rises again in the validation period to 42.83%.

PERIODS	BUY	SELL	HOLDS	TOTAL TRADES	WIN RATE
TRAIN	1,881	321	1,897	2,202	49.41%
TEST	408	222	487	630	38.73%
VALIDATION	341	266	445	607	42.83%

With the following graphs, it is possible to visualize the balance of the classes across the different splits. This confirms that in the training set there is a clear imbalance, while in the test and validation sets, the balance improves, demonstrating that adjusting the alpha value effectively achieves the class weighting goal.



Model Accuracy vs Strategy Profitability Analysis

In the chart below, the model's accuracy and the profit obtained at the end of the validation period can be compared. Regarding validation accuracy, it is not ideal for a trading prediction model, as a good accuracy for a neural network is typically around 0.70 or higher. However, when analyzing the portfolio value and the returns, the model achieved solid results, reaching a gain of 63.07%.

METRICS	PROFIT
VAL ACCURACY	0.4499
PROFIT	\$1,630, 750.09

This clearly demonstrates the importance of evaluating each indicator and continuously testing and updating the models through backtesting. At first, the model may not appear highly effective for making predictions, but the backtesting results showed that it was capable of generating solid profits despite its relatively low accuracy. The key is to keep refining the models and indicators until the most effective combination is found.

Comparison

After conducting multiple trials, it becomes evident that a model with higher validation accuracy is not always the best option, it depends on several factors. First, if the validation accuracy is high but there is a large gap compared to the training accuracy, and the validation loss exceeds values around 2 or 3, the model may be overfitting and will likely perform poorly in real-world scenarios.

It is also essential to evaluate the final performance metrics, such as the Calmar Ratio, Sharpe Ratio, and others, since even if the metrics in MLflow appear strong, the backtesting results might reveal weak performance, signaling the need to update or adjust the model to achieve better outcomes.

While achieving high validation accuracy is important, as it indicates good performance on unseen data, it is not sufficient to conclude that the model is effective. Additional performance metrics must be analyzed to ensure that the model can deliver consistent and reliable results under real trading conditions.

Conclusions

Key Findings and Strategy Viability

The trading strategy developed using the CNN model demonstrated the ability to capture meaningful patterns in stock price movements. The results showed consistent profitability across all data splits, indicating that the model was able to generalize adequately to unseen market conditions. The validation period resulted in the highest profit of \$1,630,750.09, representing a 30.56% return, which shows that the model maintained stability throughout and was able to adapt outside the training data. However, the high Maximum Drawdown of 51.20% demonstrates that the strategy has considerable exposure to risk, which means that this trading strategy might struggle during periods of high market stress. Additionally, the values of Sharpe and Calmar Ratio, which are 0.38 and 0.35, indicate that the profits

obtained by the model were achieved with a relatively high level of volatility and potential downside risk.

Overall, the CNN trading strategy demonstrates potential profitability and adaptability to new conditions, but it still requires further focus on its risk management and general stability, which is why the strategy is viable but not for everyone. With this said, this strategy could be strengthened by implementing further tuning and techniques.

Model Selection and Performance Summary

The selection of the CNN model was based on its strong ability to extract local or temporary relationships and patterns, which are crucial for understanding complex market behaviors. During training, the model achieved a moderate accuracy, which initially demonstrated a limited prediction ability. However, after backtesting, this model showed that it could generate consistent and profitable trading results. This demonstrates that accuracy alone is not enough to take an informed decision, performance must also be evaluated through financial indicators.

The CNN neural network achieved a Sharpe Ratio of 0.38, Sortino Ratio of 0.69, and Calmar Ratio of 0.35, indicating that the strategy has high volatility but profitable results. Despite these moderate but not great indicators, the model demonstrated a good capacity of adaptability to different market scenarios, which is what is what is looked for when developing trading strategies. The goal is not only to maximize profits but also ensure that the model can adapt correctly to market shifts and varying conditions.

Profitability After Transaction Costs

When considering transaction costs, the profitability of a trading strategy will evidently decrease. Considering that the chosen CNN model executed multiple

trades through time, even small fees per trade can accumulate and impact the total profit of the strategy.

Although the transaction costs slightly reduced the overall gains, the strategy remained profitable across all periods. In the train period, after considering the commissions and borrow costs, the total profit was \$1,477,014.15. In the testing period, the profit decreased slightly to \$1,194,092.07, and in the validation period, the strategy achieved the highest profit of \$1,630,750.09, representing a 30.56% return.

These results demonstrate that the model is able to generate consistent profits even after considering realistic trading costs. However, the impact of commissions and borrow costs enhances the importance of really checking trade frequencies and the size of the positions, preventing these transaction costs from affecting the profitability of the strategy.

Recommended Improvements or Extensions

In terms of recommended improvements that can be implemented to enhance the trading strategy, several points can be considered. First, implementing additional risk-focused metrics, such as Value at Risk (VaR), would provide a better evaluation of potential losses for better risk management decisions, since this was a debility of this specific trading strategy. Also, periodically retraining the model with updated market data would help it learn and adapt to new patterns and changes in market behavior, especially when severe data drift is detected. Finally, performing feature engineering to refine the input features or include different relevant indicators could improve predictive performance and make the strategy more robust in distinct conditions.

Bibliography

- IBM. (s. f.). *Deep learning*. IBM Think. Recuperado de <https://www.ibm.com/es-es/think/topics/deep-learning>
- GAMCO. (s. f.). *Perceptrón multicapa (MLP)*. GAMCO. Recuperado de <https://gamco.es/glosario/perceptron-multicapa-mlp/>
- IBM. (s. f.). *Convolutional neural networks (CNNs)*. IBM Think. Recuperado de <https://www.ibm.com/mx-es/think/topics/convolutional-neural-networks>
- Data Science Bootcamp. (2021, marzo 1). *Multilayer perceptron (MLP) vs convolutional neural network in deep learning*. Medium. Recuperado de <https://medium.com/data-science-bootcamp/multilayer-perceptron-mlp-vs-convolutional-neural-network-in-deep-learning-c890f487a8f1>
- DataCamp. (s. f.). *Introduction to convolutional neural networks (CNNs)*. Recuperado de <https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>
- Investopedia. (s. f.). *Relative strength index (RSI)*. Recuperado de <https://www.investopedia.com/terms/r/rsi.asp>
- Investopedia. (s. f.). *Momentum and relative strength index*. Recuperado de <https://www.investopedia.com/investing/momentum-and-relative-strength-index/>
- Investopedia. (s. f.). *Price rate of change (ROC)*. Recuperado de <https://www.investopedia.com/terms/p/pricerateofchange.asp>
- Investopedia. (s. f.). *Exponential moving average (EMA)*. Recuperado de <https://www.investopedia.com/terms/e/ema.asp>
- Investopedia. (2001, julio 30). *Technical analysis*. Recuperado de <https://www.investopedia.com/articles/technical/073001.asp>
- Investopedia. (s. f.). *Average true range (ATR)*. Recuperado de <https://www.investopedia.com/terms/a/atr.asp>
- Investopedia. (s. f.). *Donchian channels*. Recuperado de <https://www.investopedia.com/terms/d/donchianchannels.asp>

- Investopedia. (s. f.). *On-balance volume (OBV)*. Recuperado de <https://www.investopedia.com/terms/o/onbalancevolume.asp>
- Investopedia. (s. f.). *Volume price trend (VPT) indicator*. Recuperado de <https://www.investopedia.com/terms/v/vptindicator.asp>
- Corporate Finance Institute. (s. f.). *Accumulation/distribution indicator (A/D)*. Recuperado de <https://corporatefinanceinstitute.com/resources/equities/accumulation-distribution-indicator-a-d/>
- Investopedia. (s. f.). *Money flow index (MFI)*. Recuperado de <https://www.investopedia.com/terms/m/mfi.asp>
- Corporate Finance Institute. (s. f.). *Chaikin money flow (CMF)*. Recuperado de <https://corporatefinanceinstitute.com/resources/equities/chaikin-money-flow-cmf/>
- Codecademy. (s. f.). *Min-max and z-score normalization*. Recuperado de <https://www.codecademy.com/article/min-max-zscore-normalization>
- GeeksforGeeks. (s. f.). *Adam optimizer in deep learning*. Recuperado de <https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>
- GeeksforGeeks. (s. f.). *Sparse categorical crossentropy in machine learning*. Recuperado de <https://www.geeksforgeeks.org/machine-learning/what-is-sparse-categorical-crossentropy/>
- The IoT Academy. (s. f.). *Padding in convolutional neural networks (CNNs)*. Recuperado de <https://www.theiotacademy.co/blog/padding-in-cnn/>
- Danushi DK. (2021, abril 12). *Max pooling*. *Medium*. Recuperado de <https://medium.com/@danushidk507/max-pooling-ef545993b6e4>
- GeeksforGeeks. (s. f.). *Derivative of the sigmoid function*. Recuperado de <https://www.geeksforgeeks.org/machine-learning/derivative-of-the-sigmoid-function/>
- GeeksforGeeks. (s. f.). *ELU activation function in neural network*. Recuperado de <https://www.geeksforgeeks.org/deep-learning/elu-activation-function-in-neural-network/>

- Investopedia. (s. f.). *Sharpe ratio*. Recuperado de <https://www.investopedia.com/terms/s/sharperatio.asp>
- Wall Street Prep. (s. f.). *Maximum drawdown (MDD)*. Recuperado de <https://www.wallstreetprep.com/knowledge/maximum-drawdown-mdd/>
- Corporate Finance Institute. (s. f.). *Sortino ratio*. Recuperado de <https://corporatefinanceinstitute.com/resources/wealth-management/sortino-ratio-2/>
- Corporate Finance Institute. (s. f.). *Calmar ratio*. Recuperado de <https://corporatefinanceinstitute.com/resources/career-map/sell-side/capital-markets/calmar-ratio/>